Getting started on Medicus Suite — a guide for Nick

Hi Nick. This is everything you need to go from zero to making useful changes on this repo with Claude Code. Work through it top to bottom the first time; after that it's a reference. The repo is already set up with conventions and "skills" so Claude does most of the heavy lifting — your job is to point it at the right problem and review what it does.

---

1. The two accounts you need

These are separate things and you need both.

GitHub

- You need a GitHub account, and Dave needs to add you as a **collaborator** on `davetriska02-collab/medicus-suite` (Write access is enough — you don't need Admin).
- Once Dave's added you, you'll get an email invite. Accept it. Then you can branch, push, and open pull requests.

Claude

- You need your **own** Claude subscription. There's no shared pool — you can't pay Dave for tokens or run tasks on his account. Everyone brings their own.
- **Start on Claude Pro (~£20/month).** That is genuinely fine for the kind of "light fiddling" you described. You only need the bigger Max plan (~£180) if you end up running Claude all day, every day. Start small; upgrade later if you actually hit the limits.

---

2. How you'll actually work — pick a route

Route A: Claude Code on the web (recommended to start)

Easiest by far — nothing to install.

1. Go to **claude.ai/code** and sign in with your Claude account.
2. Connect your GitHub and pick the `medicus-suite` repo.
3. Start a session. It runs in a cloud sandbox (a fresh, throwaway copy of the

repo), so you can't break anything on your own machine or on `main`.

1. Type what you want in plain English. When you're happy, ask Claude to commit

and push to a branch and open a pull request.

This is the right place to learn the ropes. Docs:

https://code.claude.com/docs/en/claude-code-on-the-web

Route B: Claude Code on your own laptop (later, once comfortable)

More powerful, lets you actually load the extension in Chrome and see your

changes live.

1. Install the Claude Code CLI (instructions at https://code.claude.com/docs).
2. Clone the repo:

```

git clone https://github.com/davetriska02-collab/medicus-suite.git

cd medicus-suite

```

1. Run `npm install` **once**. This installs the dev tooling and wires up a

pre-commit hook that auto-checks your code formatting.

1. Run `claude` inside the folder and away you go.

Either route, the repo's `CLAUDE.md` and skills load automatically, so Claude

already knows the house rules — you inherit them for free.

---

3. The golden rules (please read — this is patient-adjacent software)

This extension reads real GP clinical data, so there are a few hard lines.

Claude knows these too, but you're the human in the loop:

1. **Never commit patient data.** Nothing from real records, exports, or

screenshots of real patients goes into git — ever. If in doubt, don't.

1. **Never push straight to `main`.** Always work on your own branch and merge

via a **pull request** so Dave can review. (`main` is the live version

people actually run.)

1. **One change at a time.** Small, focused changes are easy to review and easy

to undo. Big sprawling ones aren't.

1. **Let the tests run.** The project has automated tests and CI checks. If

something goes red on your pull request, ask Claude to explain and fix it

before merging — don't merge red.

1. **When unsure, ask.** Either ask Claude to explain what it's about to do, or

ping Dave. There's no such thing as a daft question on clinical software.

---

4. Your first 20 minutes — get a feel for it

Don't start with the hard problem. Warm up:

1. Open a session (Route A) on the repo.
2. Ask: *"Give me a tour of this codebase — what does this extension do and how

is it laid out?"*

1. Ask: *"What does the patient record visualiser do and where does it live?"*
2. Ask it to make a trivial, safe change (e.g. fix a typo in a doc), commit it

to a branch, and open a pull request. Watch the whole flow happen. Then

close the PR without merging.

That round-trip — ask → change → branch → PR → review — *is* the job. Once

that feels natural, you're ready for real work.

---

5. Your actual project — GP2GP duplicate records & degraded codes

This is the thing you raised, and it's a genuinely good first project: it's well-scoped, high-value, and it doesn't touch the scariest part of the codebase (the live queue injection). Here's how to think about it.

The problem, restated: when a patient leaves the practice and later rejoins, GP2GP can re-import their record and — because of date mismatches — effectively **duplicate the entire history**. You also want to spot and learn from **degraded codes** (codes that lost fidelity in transfer).

Where this lives in the code:

- Record parsing/extraction logic is in `engine/` (the extractors and normalisers) and the patient record visualiser (`visualiser-core.html` + `side-panel/modules/record/`).
- "Two entries that are identical except for a shifted date" is a **duplicate-detection rule**: walk the record, group entries that match on everything-except-date, flag the suspected duplicates.
- "Learning from degraded codes" is **pattern-matching over the parsed record** — which is exactly the shape of work the existing rules engine already does for drugs and QOF.

A sensible first task (don't try to do all of it at once):

> A pass that takes one already-extracted patient record and flags suspected GP2GP duplicates — entries that are the same except for a date shift — producing a simple list of "these N items look duplicated." No live-queue stuff, no auto-deleting anything. Just *detect and report* first.

Detect-and-report first, automate later. Never auto-delete clinical data — a human always confirms.

How to brief Claude on it — paste something like:

> *"I want to add a check that detects suspected GP2GP duplicate entries in a patient record — cases where a patient left and rejoined and the history got re-imported with shifted dates, so the same entry appears twice. Start by showing me where records are parsed in `engine/`, then propose where a duplicate-detection pass would slot in. Detect and report only for now — don't change or delete anything. Walk me through your plan before writing code."*

Asking it to **plan before coding** is the single best habit you can build.

Read the plan, push back on anything that smells off, *then* let it build.

---

6. Handy things to know

- **You don't need to know JavaScript** to be useful. You know the clinical

reality and the data — that's the bit Claude can't get right on its own.

Describe the problem precisely; let it write the code; you sanity-check the

result.

- **Claude can run the tests for you** — just ask "run the tests" and it will.
- **If a change goes wrong**, nothing is permanent until it's merged to `main`.

Branches and PRs are cheap and disposable. Experiment freely.

- **Stuck?** Ask Claude *"explain what you just did and why"* in plain English.

It's a patient teacher.

Welcome aboard. Start with section 4, then come find Dave for the "how to

approach it" chat on the GP2GP cases.